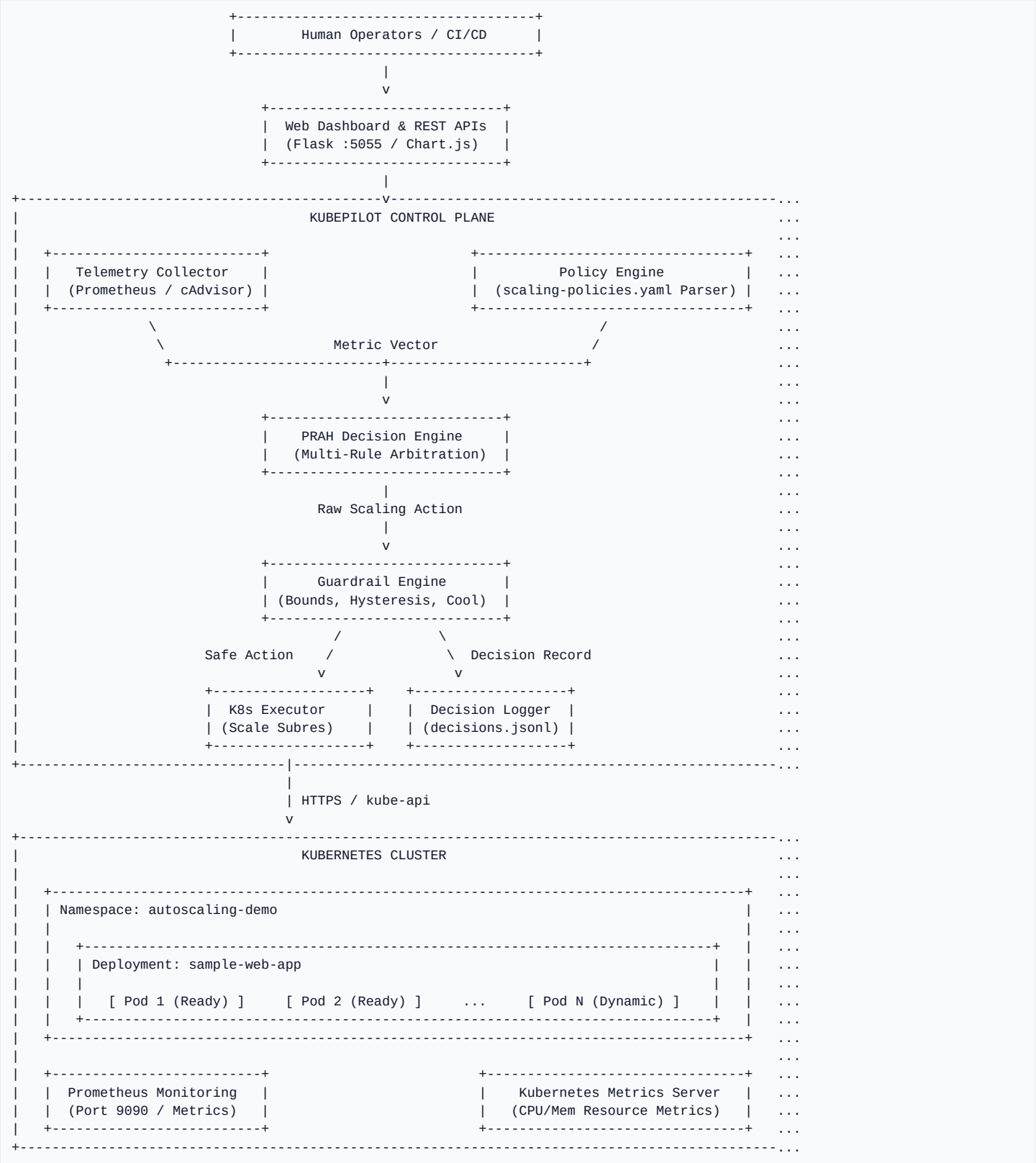
System Architecture: KubePilot Autoscaling Framework

1. Architectural Overview

KubePilot implements an autonomous, closed-loop control system that monitors, decides, validates, and executes pod scaling operations for Kubernetes workloads. The framework decouples policy definition, metric collection, decision arbitration, and Kubernetes cluster mutation into distinct, loosely-coupled modules.



2. Component Topology

2.1 Telemetry Collector (`controller/telemetry.py`)

Responsible for continuous telemetry ingestion.

- Queries Prometheus via standard HTTP API (**`/api/v1/query`**) using vector PromQL expressions for CPU utilization, memory usage, request rate, p95 latency, and 5xx error rate.
- Includes dynamic fallback to **`kubernetes.client.CustomObjectsApi`** for raw cluster metrics when Prometheus queries time out.
- Implements a deterministic synthetic mock generator when running in offline development or test environments.

2.2 Policy Engine (`controller/policy_engine.py`)

Manages declarative scaling rules loaded from **`policies/scaling-policies.yaml`**.

- Validates rule syntax, target metrics, comparison operators (**`>`**, **`>=`**, **`<`**, **`<=`**), thresholds, priority weights, and target replica calculations.
- Evaluates incoming telemetry vectors against all active policies, identifying triggered conditions and calculating recommended replica counts per rule.

2.3 PRAH Decision Controller (`controller/decision_controller.py`)

Executes the **Policy-Driven Reactive-Arbitration Heuristic (PRAH)**:

- Ingests recommendations from all triggered policies.
- In the event of competing recommendations (e.g., CPU suggests scale up to 4 while Memory suggests scale up to 3), PRAH prioritizes based on policy priority weight, severity magnitude, and safety bias.
- Resolves conflict towards the maximum safe target for scale-up requests (fail-safe for availability) and conservative evaluation for scale-down requests.
- Produces an unconstrained scaling recommendation tagged with confidence score and trigger rationale.

2.4 Guardrail Engine (`controller/guardrails.py`)

Enforces cluster stability, operational constraints, and economic bounds:

- **Absolute Bounds:** Enforces **`min_replicas: 2`** (preventing single-pod SPOF) and **`max_replicas: 10`** (preventing cluster resource exhaustion).
- **Step-Rate Limits:** Enforces **`max_scale_up_step: 3`** and **`max_scale_down_step: 1`** to ensure controlled transitions.
- **Cooldown Timers:** Implements asymmetric cooldown periods—60 seconds after scale-up, 180 seconds after scale-down—preventing thrashing.
- **Hysteresis Smoothing:** Maintains a 5-sample sliding window of recent metrics to prevent transient spikes from triggering premature scaling.
- **Flapping Detection:** Flags rapid directional reversals within a configurable observation window.

2.5 Kubernetes Executor (`controller/kubernetes_executor.py`)

Performs authenticated mutations against the Kubernetes API:

- Dynamically detects cluster configuration: in-cluster service account tokens or local kubeconfig (**`~/.kube/config`**).
- Employs resilient auto-reconnection logic with exponential backoff if the cluster restarts or connection drops.
- Scales deployments cleanly via the **`AppsV1Api.patch_namespaced_deployment_scale`** subresource.
- Queries real-time pod operational status (**`Running`**, **`Ready`**, **`Pending`**, **`CrashLoopBackOff`**).

2.6 Strategy Coordinator (`controller/strategy_coordinator.py`)

Orchestrates the continuous reconciliation loop:

- Runs periodically (default interval: 10 seconds) on a managed background worker thread.
- Sequentially coordinates: Telemetry Ingestion -> Policy Evaluation -> PRAH Arbitration -> Guardrail Enforcement -> Cluster Actuation -> Audit Logging.
- Exposes clean control methods (**`start()`**, **`stop()`**, **`evaluate_once()`**) for embedding within both standalone daemon mode and the web dashboard.

2.7 Decision Logger (`controller/decision_logger.py`)

Maintains an immutable, append-only audit trail:

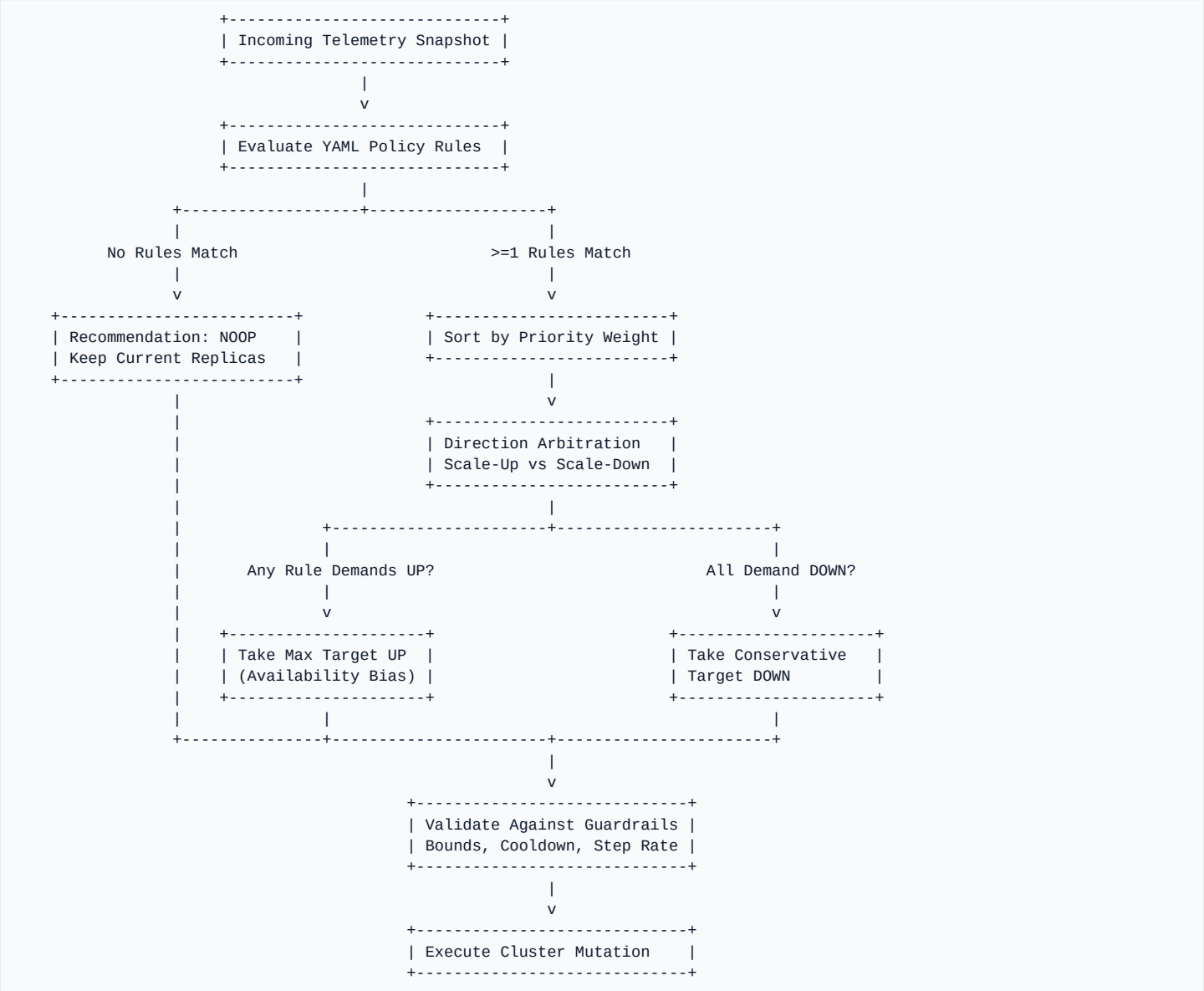
- Writes structured JSON lines to **`logs/decisions.jsonl`**.
- Captures timestamp, current replicas, recommended replicas, final approved replicas, applied status, cooldown state, triggered policy names, and instantaneous telemetry snapshot.

3. Data Pipeline & Scaling Reconciliation Loop

The following sequence diagram illustrates the lifecycle of a single reconciliation cycle:

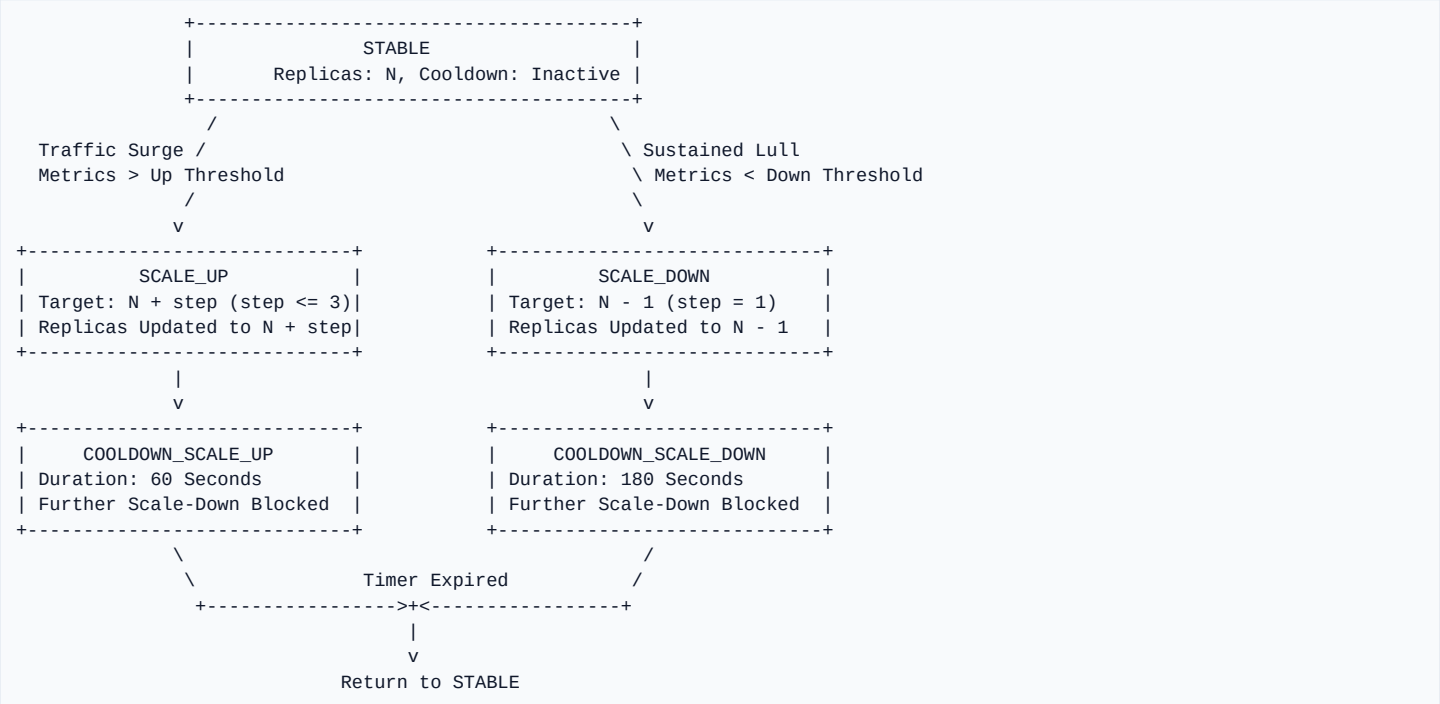
```
sequenceDiagram
    autonumber
    participant SC as StrategyCoordinator
    participant TM as TelemetryManager
    participant PE as PolicyEngine
    participant DC as DecisionController (PRAH)
    participant GE as GuardrailEngine
    participant KE as KubernetesExecutor
    participant DL as DecisionLogger
    participant K8s as Kubernetes API

    SC->>TM: get_current_metrics()
    TM-->>SC: TelemetrySnapshot(cpu=82%, mem=45%, latency_p95=210ms)

    SC->>PE: evaluate_policies(metrics)
    PE-->>SC: [TriggeredRule(HighCPU, target=4), TriggeredRule(HighLatency, target=5)]

    SC->>DC: arbitrate_decision(rules, current_replicas=2)
    DC-->>SC: Decision(target=5, reason="HighLatency priority", confidence=0.92)

    SC->>GE: validate_and_constrain(decision, current_replicas=2)
    Note over GE: Checks min/max bounds (2..10)<br/>Checks step limit (+3 max -> 5 ok)<br/>Chec...
    GE-->>SC: GuardrailApproval(target=5, approved=True)

    SC->>KE: scale_deployment("sample-web-app", replicas=5)
    KE->>K8s: PATCH /apis/apps/v1/.../scale {"spec": {"replicas": 5}}
    K8s-->>KE: 200 OK (replicas=5)
    KE-->>SC: ExecutionResult(success=True)

    SC->>DL: log_decision(decision, metrics, applied=True)
    DL-->>SC: Persisted to decisions.jsonl
```

4. PRAH Decision Arbitration Flow



5. State Machine: Cooldown and Hysteresis

The Guardrail Engine tracks operational state transitions across time to eliminate rapid scaling oscillation:



Implementation Details: KubePilot Codebase Walkthrough

1. Directory Structure

The KubePilot codebase is organized into cleanly separated domains:

```
c:\Users\SRIRAM\design_project\
■■■ controller/                # Autoscaling Control Plane & Engine
■   ■■■ __init__.py            # Module exports
■   ■■■ config.py              # Central configuration and environment parsing
■   ■■■ decision_controller.py  # PRAH decision arbitration engine
■   ■■■ decision_logger.py      # Append-only JSONL decision audit logger
■   ■■■ experiment_runner.py    # Benchmark runner (Baseline HPA vs KubePilot)
■   ■■■ guardrails.py           # Safety limits, cooldown, and hysteresis engine
■   ■■■ kubernetes_executor.py  # Resilient Kubernetes API client and mutator
■   ■■■ main.py                 # Standalone controller entrypoint daemon
■   ■■■ policy_engine.py        # Declarative YAML policy parser and evaluator
■   ■■■ strategy_coordinator.py # Master reconciliation loop and scheduler
■   ■■■ telemetry.py            # Multi-source metric collector (Prometheus / K8s)
■■■ dashboard/                 # Web Interface & Operational REST APIs
■   ■■■ app.py                  # Flask application server and route definitions
■   ■■■ requirements.txt        # Dashboard runtime dependencies
■   ■■■ static/                 # CSS styling, JavaScript graphs, and branding
■   ■■■ templates/              # Jinja2 HTML templates for all views
■■■ policies/                  # Declarative Autoscaling Configuration
■   ■■■ scaling-policies.yaml   # Canonical scaling rules and guardrail parameters
■■■ monitoring/                # Telemetry & Observability Definitions
■   ■■■ prometheus.yml          # Local Prometheus scrape configuration
■   ■■■ prometheus-config.yaml  # In-cluster ConfigMap manifest
■   ■■■ prometheus-deployment.yaml # In-cluster Prometheus deployment & service
■   ■■■ prometheus-values.yaml  # Helm values configuration
■■■ k8s/                       # Kubernetes Infrastructure Manifests
■   ■■■ namespace.yaml          # Definition for `autoscaling-demo` namespace
■   ■■■ baseline-hpa.yaml       # Native Kubernetes HorizontalPodAutoscaler
■   ■■■ controller-rbac.yaml    # ClusterRole and ServiceAccount for controller
■   ■■■ controller-deployment.yaml # Production Deployment manifest for KubePilot
■   ■■■ dashboard-deployment.yaml  # Production Deployment manifest for Dashboard
■   ■■■ policies-configmap.yaml # ConfigMap mount for scaling-policies.yaml
■■■ app/                       # Benchmark Target Workload Application
■   ■■■ app.py                  # Flask mock workload with CPU burn endpoints
■   ■■■ Dockerfile              # Container image definition
■   ■■■ requirements.txt        # Workload Python dependencies
■   ■■■ sample-app.yaml         # Kubernetes Deployment & Service manifests
■■■ experiments/               # Experiment Benchmark Artifacts & Output
■   ■■■ results/                # CSV and JSON empirical run data
■■■ tests/                     # Automated Regression & Unit Test Suite
■   ■■■ test_telemetry.py       # Metrics collection & fallback tests
■   ■■■ test_policy_engine.py   # Policy parsing and evaluation tests
■   ■■■ test_decision_controller.py # PRAH arbitration unit tests
■   ■■■ test_guardrails.py      # Boundary and rate-limit safety tests
■   ■■■ test_hysteresis.py      # Window smoothing and flapping tests
■   ■■■ test_kubernetes_executor.py # K8s client and scale patch tests
■   ■■■ test_dashboard_api.py   # Web API endpoint validation tests
■   ■■■ test_experiments.py     # Workload generator & runner tests
■   ■■■ test_fixes_regression.py # Live bug regression verification
■■■ documents/                 # Authoritative Project Documentation Suite
    ■■■ markdown/              # Comprehensive reference manuals
    ■■■ pdf/                   # Published executive and technical PDFs
    ■■■ archive/               # Historical review records and previous reports
```

2. Core Modules & Class Reference

2.1 `controller.config.Config`

The **`Config`** class centralizes all runtime configuration, pulling from environment variables with safe fallback defaults:

- **`KUBECONFIG_PATH`**: Path to the kubeconfig file (defaults to **`~/.kube/config`** or in-cluster token).
- **`K8S_NAMESPACE`**: Target Kubernetes namespace (**`autoscaling-demo`**).
- **`TARGET_DEPLOYMENT`**: Target Deployment resource name (**`sample-web-app`**).
- **`PROMETHEUS_URL`**: Prometheus API endpoint (**`http://localhost:9090`** or **`http://prometheus:9090`**).
- **`EVALUATION_INTERVAL`**: Polling cycle frequency in seconds (default: **`10`**).
- **`MIN_REPLICAS`**: Hard floor for pod count (strictly set to **2** to guarantee high availability).

- **`MAX_REPLICAS`**: Hard ceiling for pod count (default: **`10`**).
- **`COOLDOWN_SCALE_UP`**: Minimum seconds between scale-up actions (default: **`60`**).
- **`COOLDOWN_SCALE_DOWN`**: Minimum seconds between scale-down actions (default: **`180`**).
- **`DECISION_LOG_PATH`**: Destination path for JSONL audit trail (**`logs/decisions.jsonl`**).

2.2 `controller.telemetry.TelemetryCollector`

Collects instantaneous workload metrics from Prometheus and cluster APIs.

- **`get_metrics() -> Dict[str, Any]`**:

Queries Prometheus with PromQL expressions:

- CPU: **`sum(rate(container_cpu_usage_seconds_total{namespace="autoscaling-demo",pod=~"sample-web-app-.*"}[1m])) / sum(kube_pod_container_resource_limits{resource="cpu"}) * 100`**
- Memory: **`sum(container_memory_working_set_bytes{namespace="autoscaling-demo",pod=~"sample-web-app-.*"}) / sum(kube_pod_container_resource_limits{resource="memory"}) * 100`**
- Request Rate: **`sum(rate(http_requests_total{job="sample-app"}[1m]))`**
- P95 Latency: **`histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket{job="sample-app"}[1m])) by (le))`**
- **`_get_kubernetes_fallback_metrics() -> Dict[str, Any]`**:

Directly accesses **`metrics.k8s.io`** if Prometheus is unreachable.

- **`_get_mock_metrics() -> Dict[str, Any]`**:

Generates deterministic, continuous synthetic metrics when cluster connection is offline.

2.3 `controller.policy_engine.PolicyEngine`

Parses and evaluates declarative YAML rules defined in **`policies/scaling-policies.yaml`**.

- **`load_policies(policy_path: str)`**:

Reads the YAML definition, constructs **`PolicyRule`** objects, and validates conditional operators (**`>`**, **`>=`**, **`<`**, **`<=`**).

- **`evaluate(metrics: Dict[str, Any], current_replicas: int) -> List[PolicyRecommendation]`**:

Iterates across loaded policies. For each matched condition, calculates recommended replicas using either absolute targets, relative offsets (**`+2`**, **`-1`**), or proportional scale equations:

$$\text{target} = \left\lceil \text{current} \times \left(\frac{\text{observedValue}}{\text{thresholdValue}} \right) \right\rceil$$

- **`reset()`**:

Clears loaded policies and resets engine state (critical for dynamic reloads and unit testing).

2.4 `controller.decision_controller.DecisionController`

Implements the **PRAH** algorithm:

- **`arbitrate(recommendations: List[PolicyRecommendation], current_replicas: int) -> Decision`**:
- Sorts recommendations by priority weight (1 to 100).
- If both scale-up and scale-down rules trigger simultaneously:
- Selects the **scale-up** action to prioritize service availability and SLA protection.
- Chooses the highest replica target among competing scale-up rules.
- If only scale-down rules trigger:
- Selects the conservative scale-down target to prevent undershoot.
- Calculates a composite confidence score based on condition margin and rule weight:

$$\text{confidence} = \min\left(1.0, 0.5 + 0.5 \times \frac{|\text{observed} - \text{threshold}|}{\text{threshold}}\right)$$

2.5 `controller.guardrails.GuardrailEngine`

Applies strict operational guardrails before cluster mutation:

- **`validate_decision(decision: Decision, current_replicas: int) -> GuardrailResult`**:

1. **Floor & Ceiling Bounds:** Enforces *text{target} *in [*text{min_replicas}, *text{max_replicas}]* (e.g. *[2, 10]*).
2. **Step Limit:**

- Scale up: *Delta R *le *text{max_scale_up_step}* (default: **+3**).
- Scale down: *Delta R *ge -*text{max_scale_down_step}* (default: **-1**).

3. **Cooldown Window:**

- If scaling up and elapsed time < *text{cooldown_scale_up}* (60s), rejection with **`COOLDOWN_ACTIVE`**.
- If scaling down and elapsed time < *text{cooldown_scale_down}* (180s), rejection with **`COOLDOWN_ACTIVE`**.

4. **Hysteresis Smoothing:** Requires metric thresholds to persist across at least 3 out of 5 consecutive evaluation ticks before approving a scale-down mutation.
5. **Flapping Suppression:** Disallows scale-down if a scale-up occurred within the last 120 seconds.

2.6 `controller.kubernetes_executor.KubernetesExecutor`

Interacts with the Kubernetes API server using the official Python client.

- **`scale_deployment(replicas: int) -> bool`**:

Invokes **`AppsV1Api.patch_namespaced_deployment_scale(name, namespace, {"spec": {"replicas": replicas}})`**.

- **`get_deployment_status() -> DeploymentStatus`**:

Queries desired replicas, ready replicas, available replicas, and current pod names.

- **`get_pod_details() -> List[PodInfo]`**:

Queries pod status phase, ready status, restart count, node name, and pod IP.

- **`ensure_connection()`**:

Detects client disconnects and cleanly recreates API sessions without crashing the controller.

2.7 `controller.strategy_coordinator.StrategyCoordinator`

Coordinates the autonomous reconciliation loop.

- Runs on a dedicated daemon background thread.
- In each tick:

1. Polls **`TelemetryCollector.get_metrics()`**.
2. Queries current replica count from **`KubernetesExecutor`**.
3. Evaluates active policies via **`PolicyEngine`**.
4. Arbitrates decision via **`DecisionController`**.
5. Validates decision via **`GuardrailEngine`**.
6. Executes scale patch via **`KubernetesExecutor`** if approved and target differs from current.
7. Logs structured decision via **`DecisionLogger`**.

- Supports on-demand single execution via **`evaluate_once()`** for API triggers.

2.8 `controller.experiment_runner.ExperimentRunner`

Executes synthetic benchmark workloads against **`sample-web-app`**:

- **`run_experiment(experiment_type: str, mode: str, duration_seconds: int)`**:
- Supports **`stable_workload`** (constant 30 req/s) and **`traffic_spike`** (spike up to 120 req/s).
- Executes across **`baseline`** (native HPA) and **`proposed`** (KubePilot PRAH).
- Records time-series samples of timestamp, requests/sec, CPU%, latency p95, desired replicas, and ready replicas.
- Exports results simultaneously to **`experiments/results/*.csv`** and **`experiments/results/*.json`**.

3. Web Dashboard Architecture (`dashboard/app.py`)

The dashboard is built on Flask and TailwindCSS, exposing operational views and REST endpoints:

- **`/` (Overview):** Unified view of cluster connectivity, target deployment replicas, live telemetry, and recent scaling decisions.
- **`/workloads`:** Pod inspection, container state, restarts, and pod distribution.
- **`/telemetry`:** Real-time charts of CPU utilization, memory usage, request throughput, and p95 latency.
- **`/policies`:** Visual policy inspector showing rule definitions, metric targets, and priority weights.
- **`/decisions`:** Detailed view of PRAH arbitration logic and confidence scores.
- **`/history`:** Historical audit table of all decisions loaded from **`logs/decisions.jsonl`**.
- **`/system-health`:** Connection health, Prometheus status, and controller heartbeat.
- **`/healthz` & `/readyz`:** Kubernetes liveness and readiness probe endpoints.